Football Player Detection & Match Analytics with YOLOv8

Technical report · Shubham Tanna

Python · Ultralytics YOLOv8 · OpenCV · scikit-learn · pandas

1. Summary

This project trains a computer-vision model to find the ball, goalkeepers, outfield players and referees in broadcast football frames, then uses those detections to split players into teams and estimate possession. The full pipeline (data analysis, training, evaluation and analytics) is written in Python and runs on an ordinary CPU, with no GPU.

- **Accuracy.** A 3-million-parameter YOLOv8-nano model reached 0.83 mAP@0.5 on the held-out test set, with players at 0.99, referees at 0.95 and goalkeepers at 0.93.
- **Weak spot.** The ball is the hard case: when the model calls a ball it is right 95% of the time, but it finds only 38% of balls (mAP@0.5 0.43).
- **Cheap win.** Choosing the inference resolution on validation data (960 px instead of the 640 px used in training) lifted test mAP@0.5 from 0.79 to 0.83, and ball mAP@0.5 from 0.28 to 0.43, without retraining.
- **Analytics.** Jersey-colour clustering separates the two teams in each clip without any kit labels. A nearest-player rule then estimates possession in every frame where the ball is detected.
- **Versus v1.** This Python rebuild matches the first, platform-trained version overall (0.83 vs 0.82 mAP@0.5) and beats it on three of the four classes, and every step can now be inspected and rerun.

2. Problem and objective

Clubs and broadcasters increasingly rely on tracking data for tactics, player load and highlights. That data usually comes from expensive multi-camera systems, wearable sensors or manual tagging. Detecting players and the ball directly from the broadcast feed is the cheapest starting point, but it is hard. Players look alike and overlap, the camera pans and zooms, lighting changes, and the ball is often only a few pixels wide.

Objective: build a detector that locates and classifies every player, goalkeeper, referee and the ball in a frame, measure honestly how well it does each job, and show how its output turns into match information (teams and possession).

3. Background

A separate literature review (Literature_Review.pdf) covers three recent papers in detail. In brief:

- **Wang & Li (2022).** A lightweight CNN with a feature-pyramid design reached 0.915 AP on the ISSIA-CNR benchmark, ahead of Faster R-CNN, but struggled with small and occluded players.
- **Merzah et al. (2024).** Classical OpenCV (CSRT) tracking turned player positions into speed and distance without GPS hardware, but every player had to be selected by hand.
- **Zhao (2024).** An upgraded YOLOv5 raised mAP@0.5 from 0.81 to 0.89 while nearly halving compute, and it detected the ball as well as players. Its stricter mAP@0.5:0.95 did not improve.

Takeaways for this project: use a compact single-stage YOLO detector, keep the ball as its own class and report it separately, and build analytics on top of the detections.

4. Data

The data is the public "football-players-detection" dataset on Roboflow Universe (version 12, CC BY 4.0). It contains 372 frames, each 1280×1280, sampled from 56 short clips of Bundesliga broadcast footage, with bounding boxes for four classes.

Split	Frames	Objects
Train	298	7,133
Validation	49	1,174
Test	25	598
Total	**372**	**8,905**

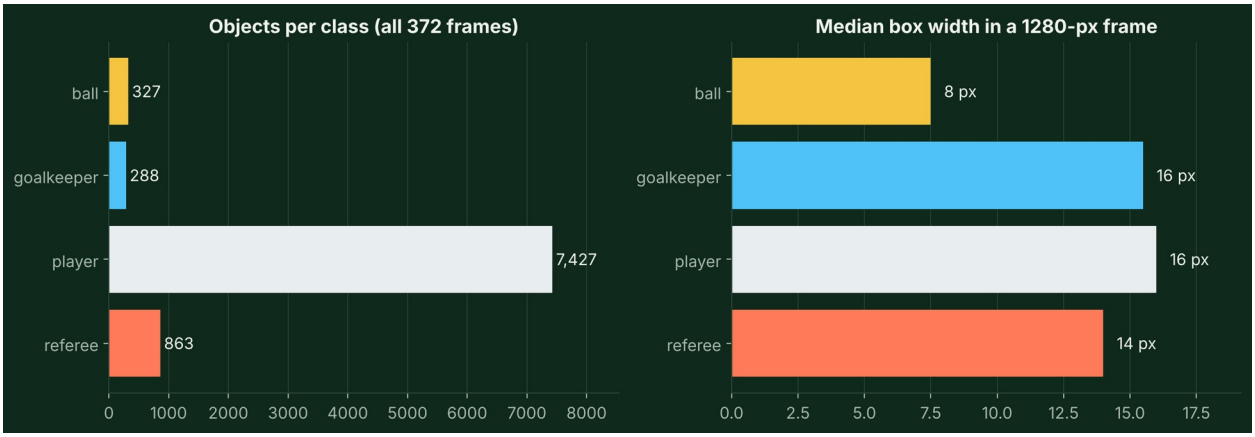


Figure 1. Class counts and median box width. Players make up 83% of labels; the median ball is 8 px wide.

Two features of the data shape everything that follows:

- **Imbalance.** Players outnumber balls 23 to 1, and there is at most one ball per frame, so the model sees far fewer examples of the ball.
- **Scale.** The median ball is 8 px wide in a 1280-px frame, or 4 px once the frame is resized to 640 px for training. Small objects are the classic weakness of detectors at this scale.

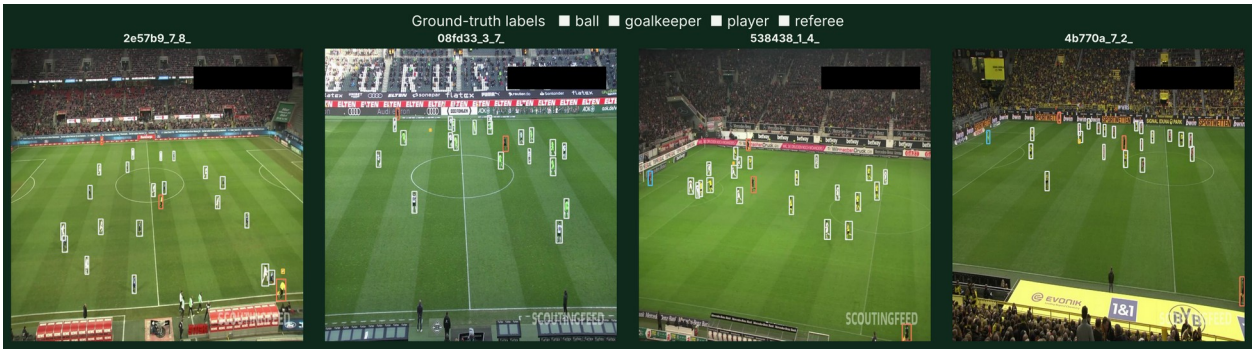


Figure 2. Sample training frames with ground-truth labels.

Caveat on the split: frames from the same clip appear in the training, validation and test sets. Test frames are therefore similar to frames the model has seen, and the test scores below should be read as an upper bound on performance on completely new matches.

5. Method

5.1 Model and training

- **Model.** YOLOv8n, the smallest YOLOv8 variant (3.0 M parameters, 8.6 GFLOPs), chosen so the whole project could be trained and run on a laptop-class CPU.
- **Transfer learning.** Started from COCO-pretrained weights, so the network already recognises "person" and "sports ball", and fine-tuned on the 298 training frames.
- **Settings.** 640 px input, batch size 8, Ultralytics default augmentation (mosaic, HSV colour jitter, flips, scale), and a fixed budget of 1.75 hours on 2 CPU cores, which came to 43 epochs. The checkpoint with the best validation score was kept.

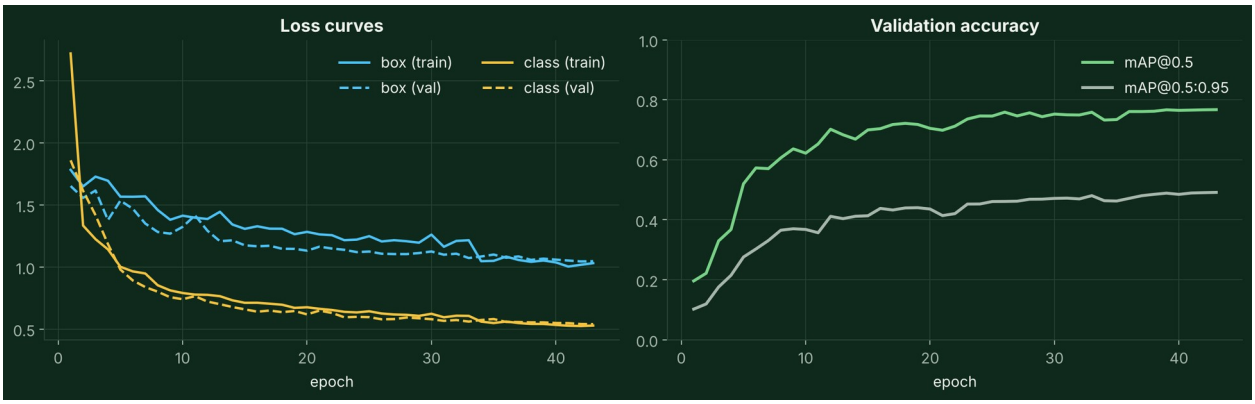


Figure 3. Loss curves (left) and validation accuracy (right) over 43 epochs.

Validation mAP@0.5 passed 0.5 by epoch 5 and was still rising slowly at the end, so a longer run would likely add a little more accuracy. Training and validation losses track each other closely, which shows no sign of overfitting.

5.2 Choosing the inference resolution

A model trained at 640 px can be given larger images at prediction time. More pixels should help the ball, but objects that look larger than anything seen in training can hurt the other classes. Three sizes were compared on the validation set only:

Inference size	mAP@0.5 (all)	mAP@0.5 (ball)	Time per frame (CPU)
640 px	0.768	0.287	113 ms
960 px	0.806	0.432	283 ms
1280 px	0.710	0.325	540 ms

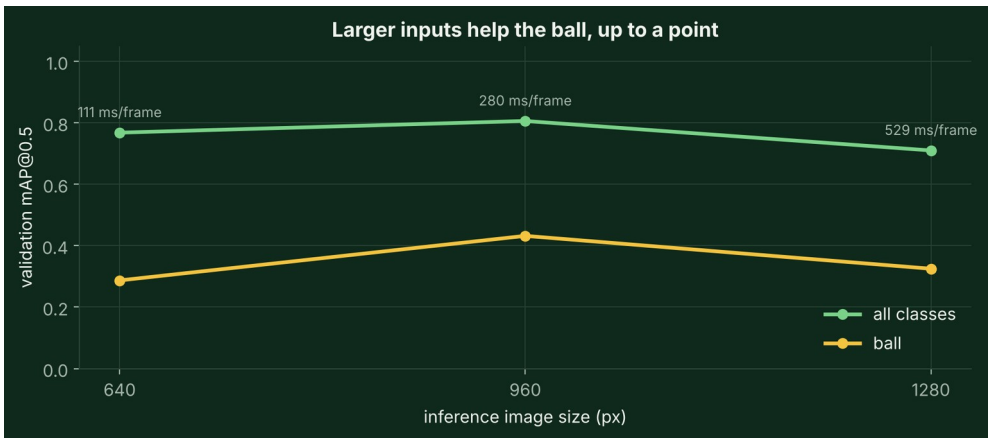


Figure 4. Validation accuracy by inference size.

960 px was selected: it lifts the ball by 0.15 mAP and the other classes slightly, at about 2.5× the inference time. At 1280 px, players and goalkeepers fall outside the size range the model learned, and accuracy drops.

5.3 Analytics layer

- **Team identification.** For each detected player, the median colour (in Lab colour space) of a small patch at the centre of the shirt is computed. Using only the centre keeps grass out of the sample without filtering by colour, which matters because some teams wear green. Within each clip, k-means with k = 2 splits the players into two teams. No kit labels are needed.
- **Possession estimate.** In each frame, the ball detection with the highest confidence is compared with the feet (bottom-centre of the box) of every player. The nearest player's team is credited with possession if that player is within 1.5 body-heights of the ball; otherwise the frame is marked "loose ball". Frames with no ball detected are counted separately.

6. Results

6.1 Test-set accuracy

The test set (25 frames, 598 objects) was not used for training, model selection or the resolution choice.

Class	Objects	Precision	Recall	mAP@0.5	mAP@0.5:0.95
ball	24	0.95	0.38	0.43	0.18
goalkeeper	19	0.94	0.78	0.93	0.72
player	499	0.98	0.97	0.99	0.79
referee	56	0.93	0.88	0.95	0.63
all	**598**	**0.95**	**0.75**	**0.83**	**0.58**

- **People are solved.** Players (0.99), referees (0.95) and goalkeepers (0.93) are detected almost perfectly.
- **The ball is not.** The ball has high precision (0.95) but low recall (0.38). The model is conservative: it rarely invents a ball but misses most real ones, mainly when the ball is tiny, blurred or partly hidden by a player.
- **Speed.** At 960 px the model processes about 4 frames per second on a 2-core CPU (≈9 fps at 640 px). On a modest GPU, the same model would run comfortably in real time.

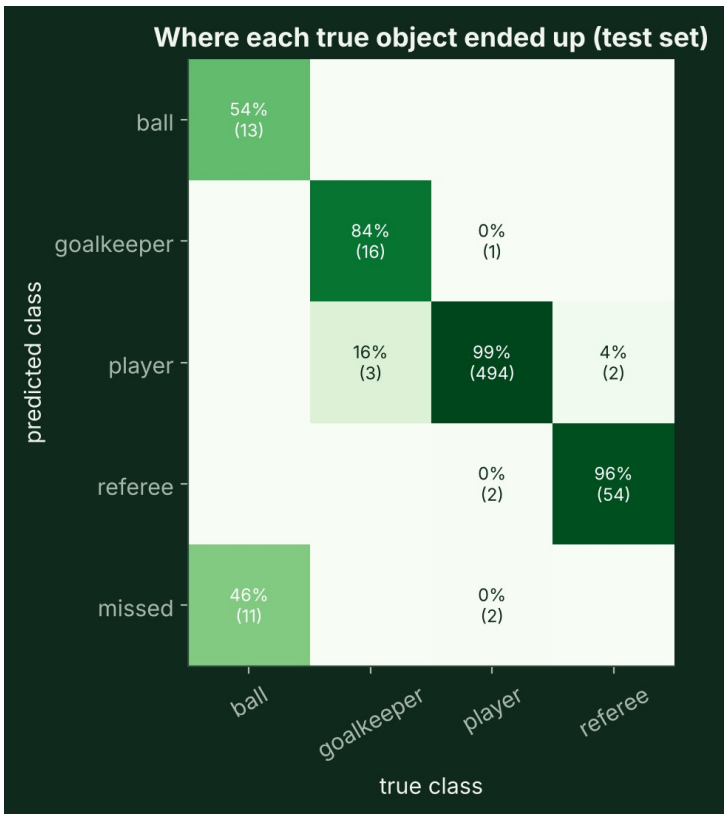


Figure 5. Where each true test object ended up. The "missed" row counts objects the model did not detect.

The confusion matrix shows the classes are rarely mixed up with each other. The main confusion is 3 of 19 goalkeepers labelled as players, which is unsurprising since they differ only by kit. Nearly all errors are missed objects rather than wrong labels, and 11 of the 24 balls account for most of them.



Figure 6. Ground truth (top) vs model predictions (bottom) on three test frames.

6.2 Comparison with version 1

The first version of this project was trained on Roboflow's hosted platform (Roboflow 3.0 Object Detection, Fast). Its scores come from the Roboflow dashboard and its own validation split, so the comparison is indicative rather than controlled.

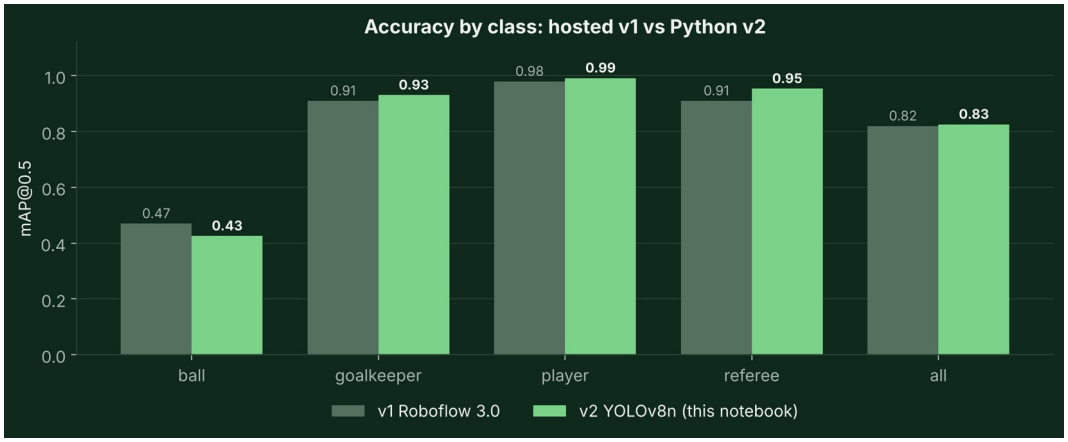


Figure 7. mAP@0.5 by class: hosted v1 vs Python v2.

Version 2 matches v1 overall (0.83 vs 0.82) and is ahead on goalkeepers, players and referees. It is slightly behind on the ball (0.43 vs 0.47), which is consistent with v1's much longer training run on the platform. The real gain is transparency: every step is now code that can be inspected, changed and rerun.

7. Match analytics

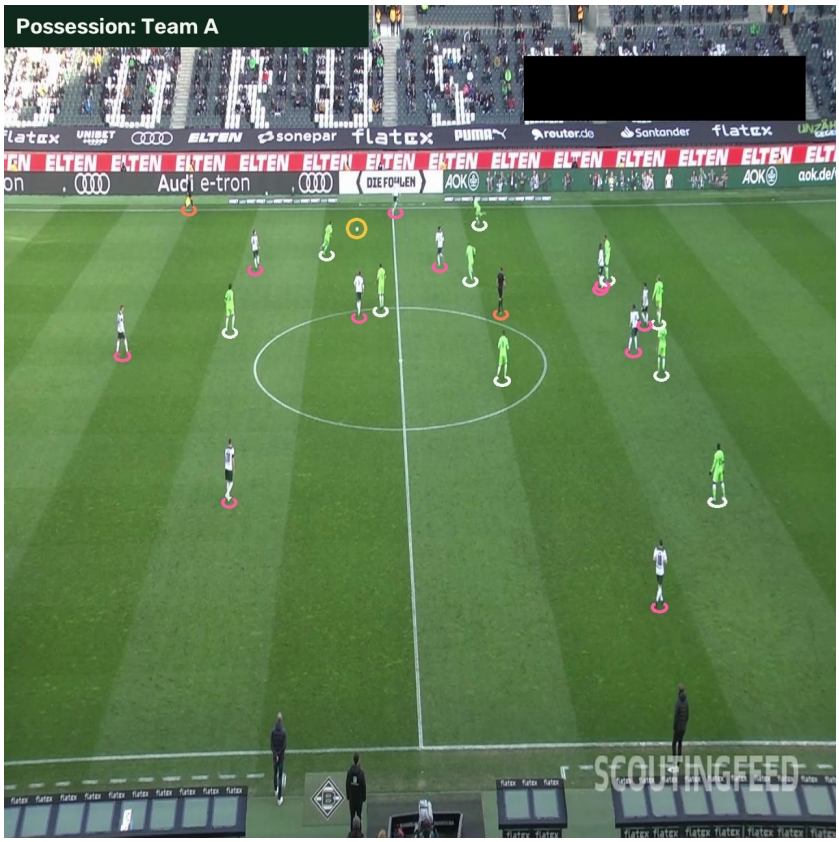


Figure 8. Annotated output: team A (white rings), team B (pink), referees (orange), ball (yellow). The banner shows the possession call.

Team clustering produced a median of 10 and 11 players per team per frame, close to the expected 10 outfield players a side visible in a wide shot. Across all 372 frames, possession was credited to one of the two teams in 40% of frames, loose ball in 8%, and in 53% no ball was detected.

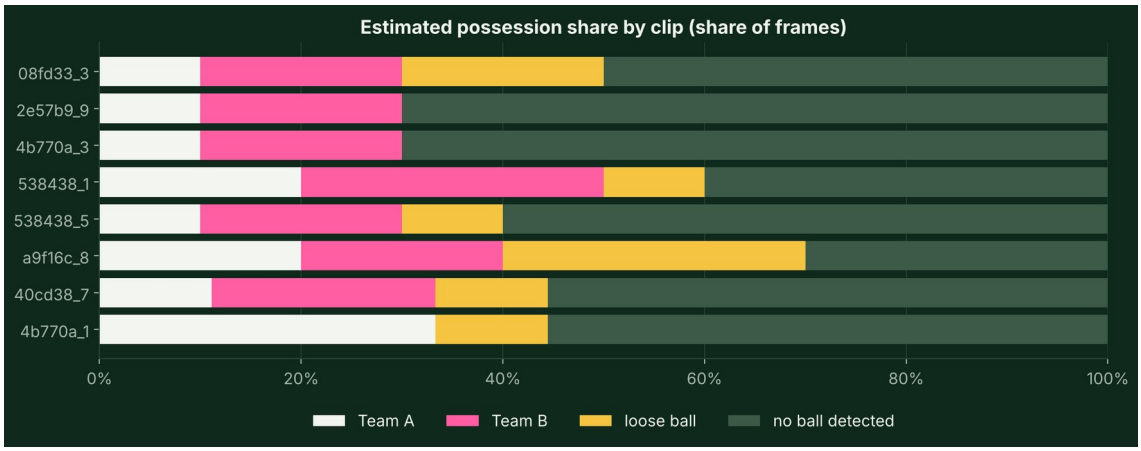


Figure 9. Estimated possession share for the eight clips with the most frames.

So what: the detector is good enough for player-level analytics (team shape, spacing, counts) today. Ball-dependent metrics such as possession are capped by ball recall: improving ball detection is the single biggest lever for the analytics layer.

8. Limitations and next steps

- **Optimistic test scores.** Frames from the same clips appear in train and test sets. A stricter evaluation would hold out entire matches.
- **Ball recall.** Train at 1280 px on a GPU, try a larger model (YOLOv8s/m), or use tiled inference (e.g. SAHI) so the ball is not shrunk.
- **Possession is a proxy.** Possession counts frames, not seconds, and the nearest player is not always the one in control. Adding a tracker (ByteTrack) and a pitch homography would give possession over time, distance covered and heatmaps in metres.
- **Team clustering.** k-means only sees shirt colour, so it can mis-assign players in shadow or with similar kits. Goalkeepers and referees are detected as separate classes for this reason.

9. Reproducibility

Everything in this report is produced by notebook/Football_Player_Detection_YOLOv8.ipynb. Trained weights are in runs/yolov8n_football/weights/best.pt, and src/analyse_match.py runs the full detection-and-analytics pipeline on any video or folder of frames. Random seed: 42. Software: Ultralytics 8.4, PyTorch 2.x (CPU), OpenCV, scikit-learn.

References and credits

Wang, T. & Li, T. (2022). Deep Learning-Based Football Player Detection in Videos. Computational Intelligence and Neuroscience, 2022, 3540642. https://doi.org/10.1155/2022/3540642

Merzah, B.M., Croock, M.S. & Rashid, A.N. (2024). Football player tracking and performance analysis using the OpenCV library. Mathematical Modelling of Engineering Problems, 11(1), 123–132. https://doi.org/10.18280/mmep.110113

Zhao, K. (2024). Enhancing the Performance and Accuracy in Real-Time Football and Player Detection Using Upgraded YOLOv5 Architecture. International Journal of Computational Intelligence Systems, 17, 163. https://doi.org/10.1007/s44196-024-00565-x

Jocher, G., Chaurasia, A. & Qiu, J. (2023). Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics (AGPL-3.0)

Dataset: "football-players-detection", version 12, Roboflow Universe (workspace roboflow-jvuqo). Licensed CC BY 4.0.
https://universe.roboflow.com/roboflow-jvuqo/football-players-detection-3zvbc